

C 语言程序设计大作业

Codeforces Crawler 2025

单 位：计算机学院

班 级：计算机科学与技术一班

学 号：202505486219

姓 名：赵开

任课教师：谢勇

湘潭大学

2025 年 5 月

文档信息	状态
源代码	☐ 有
可执行文件	☐ 有
自评成绩	优
答辩PPT	☐ 有
设计文档	☐ 有

学术诚信承诺书

赵开，学号 202505486219，系信息工程学院计算机系 24 级计算机科学与技术专业 1 班学生。本人承诺本课程设计提交的所有源代码和文档不存在抄袭、剽窃行为。如果被他人发现存在抄袭、剽窃、雷同等学术不端行为，愿意承担所带来的一切后果。

赵开

2025 年 5 月

目录

1 题目概述

1.1 概述

1.2 所要完成的任务说明

1.3 完成情况

1.4 开发环境说明

2 程序分析与设计

2.1 函数详细说明

2.2 函数调用关系图

3 系统演示

3.1 系统各功能运行截图与说明

4 设计体会

4.1 设计过程中的问题及其解决方案

4.2 设计过程中的错误及其原因分析

4.3 体会和收获

1 题目概述

1.1 概述

本项目旨在设计并实现一个功能完善的 Codeforces 数据爬取与可视化分析系统。该程序的核心功能是利用 Codeforces 官网提供的官方 API，自动化地抓取指定用户的公开比赛信息。通过对原始数据进行一系列的处理、分析和统计，程序最终会将结果以结构化数据的形式输出，并进一步利用这些数据生成动态、交互式的可视化图表，以使用户能够在浏览器中直观地回顾和分析其在 Codeforces 平台上的比赛历程和成长轨迹。

1.2 所要完成的任务说明

根据项目要求，主要任务分为必做功能和选作功能两大类，具体如下表所示：

序号	功能分类	任务描述
B1	必做	读取和解析指定用户的基本信息（Handle、Rating、Rank、Avatar 等）
B2	必做	获取并筛选出过去一年 Codeforces 官方举办的所有 Div.1~Div.4 级别的常规比赛列表
B3	必做	读取指定用户在过去一年内实际参加的比赛列表，并筛选出 Div.1~Div.4 级别比赛
B4	必做	对于用户参加的每一场符合条件的比赛，详细读取其排名、每道题目的得分、Rating 变化及头衔变化
B5	必做	统计用户在过去一年内参加各级别比赛的总次数
B6	必做	统计用户在过去一年内参赛次数的月度分布情况
B7	必做	计算用户在过去一年内的整体出勤率（实际参赛数 / 总比赛数）
B8	必做	汇总所有参赛记录，含比赛级别、名称、时间、排名、题目得分、Rating 变化和头衔变化
B9	必做	统计并展示题目分值的分布情况
S1	选作	赛时/补题判定：区分用户每道题是在比赛期间还是赛后完成
S2	选作	AC 数量直方图：按题目 Rating 分桶，以全部、近一年、近 180 天、近 30 天四个时间窗口统计
S3	选作	生成自包含的 HTML 可视化报告，含 ECharts 交互式图表，支持多用户索引和搜索

1.3 完成情况

截至目前，项目各项功能的完成情况如下表所示。对于已完成的功能，均已实现核心逻辑并通过了基本测试。

功能序号	完成情况说明
B1	已完成。通过 cf_fetch_user_info 获取，rating_color / rating_title 映射颜色和头衔，生成 HTML 展示。
B2	已完成。通过 cf_fetch_contest_list 获取比赛列表，按名称关键词 "Div. 1~4" 筛选，限制过去一年内。
B3	已完成。通过 cf_fetch_user_rating 获取用户比赛记录，筛选过去一年内的 Div.1~4 比赛。
B4	已完成。通过 cf_fetch_contest_standings 逐场获取排名、题目得分及 Rating 变化。
B5	已完成。在 build_user_summary 中统计各级别参赛次数。
B6	已完成。按月份维度统计参赛次数。
B7	已完成。通过 B2/B3 数据比较计算得出勤率。
B8	已完成。在 HTML 详情页中以表格形式汇总展示所有参赛记录。
B9	已完成。在 HTML 中以直方图展示题目 rating 分布。
S1	已完成。通过提交时间与比赛结束时间比较，精确标记每道题的赛时/补题状态。
S2	已完成。四个时间窗口的 AC 直方图通过 cJSON 树集成到分析结果中，ECharts 渲染。
S3	已完成。generate_all_html 生成 index.html（用户列表+搜索）和 report_.html（详情页）。

1.4 开发环境说明

本项目主要在以下环境中进行开发与测试：

- 操作系统：Windows 11 + MSYS2 UCRT64 模拟 Linux 环境
- C 编译器：GCC 14.x (MSYS2 UCRT64)
- 主要依赖库：libcurl (HTTP 请求)、cJSON (JSON 解析)
- 构建工具：GNU Make / 手动 GCC 编译
- IDE：Visual Studio Code / Neovim
- 可视化：ECharts 5 (CDN 加载)
- 开发语言：C11 标准

2 程序分析与设计

2.1 函数详细说明

以下对项目中创建的所有函数进行详细说明，包含函数原型、功能、参数意义及返回值类型。共 23 个函数，按模块分组。

编号	描述详情
001	函数原型: void rate_limiter_init(RateLimiter *rl) 所属模块: rate_limiter 功能说明: 初始化限速器结构体, 将 last_request_time 设为 0.0, min_interval 设为 2.0 秒, 确保首次请求无需等待。 参数说明: RateLimiter* rl — 指向 RateLimiter 结构体的指针, 函数将初始化其内部字段 返回值: void (无返回值) 详细说明: 在程序启动时调用, 为后续的 API 请求频率控制做准备。
002	函数原型: void rate_limiter_wait(RateLimiter *rl) 所属模块: rate_limiter 功能说明: 阻塞等待, 直到距离上次请求的时间超过 min_interval (2 秒)。使用 Windows 的 QueryPerformanceCounter 或 Linux 的 clock_gettime 获取高精度时间。 参数说明: RateLimiter* rl — 指向 RateLimiter 结构体的指针, 读取上次请求时间戳 返回值: void (无返回值) 详细说明: 在每次 API 请求前调用。如果距上次请求不足 2 秒, 则通过 Sleep/usleep 休眠剩余时间, 遵守 CF API 的频率限制要求。
003	函数原型: void rate_limiter_tick(RateLimiter *rl) 所属模块: rate_limiter 功能说明: 在 API 请求完成后调用, 记录当前时间戳到 last_request_time, 供下一次 wait 判断间隔使用。 参数说明: RateLimiter* rl — 指向 RateLimiter 结构体的指针, 更新 last_request_time 字段 返回值: void (无返回值) 详细说明: 与 rate_limiter_wait 成对使用, 形成"等待→请求→记录"的完整请求控制循环。
004	函数原型: char *read_text_file(const char *path) 所属模块: json_utils 功能说明: 读取指定文本文件的全部内容, 返回 malloc 分配的字符串。使用 fseek/ftell 获取文件大小, 精确分配缓冲区。 参数说明: const char* path — 文件路径字符串 返回值: char* — 指向文件内容的 malloc 分配字符串, 失败返回 NULL 详细说明: 调用者负责 free 返回的字符串。若文件为空或打开失败, 返回 NULL。

005	函数原型: <code>int write_text_file(const char *path, const char *content)</code> 所属模块: <code>json_utils</code> 功能说明: 将字符串内容写入指定文本文件（覆盖模式）。使用 <code>fopen</code> 的 "w" 模式写入。 参数说明: <code>const char* path</code> — 目标文件路径 <code>const char* content</code> — 待写入的字符串内容 返回值: <code>int</code> — 成功返回 0, 失败返回 -1 详细说明: 用于将 API 响应写入缓存文件, 以及将生成的 HTML 写入输出文件。
006	函数原型: <code>cJSON *read_json_file(const char *path)</code> 所属模块: <code>json_utils</code> 功能说明: 读取 JSON 文件并解析为 cJSON 对象树。先调用 <code>read_text_file</code> 读取文本, 再用 <code>cJSON_Parse</code> 解析。 参数说明: <code>const char* path</code> — JSON 文件路径 返回值: <code>cJSON*</code> — 指向解析后的 cJSON 对象树的指针, 失败返回 NULL 详细说明: 调用者负责用 <code>cJSON_Delete</code> 释放返回的对象。用于读取缓存的 CF API 响应。
007	函数原型: <code>int write_json_file(const char *path, const cJSON *json)</code> 所属模块: <code>json_utils</code> 功能说明: 将 cJSON 对象序列化为格式化的 JSON 字符串并写入文件。使用 <code>cJSON_Print</code> 生成带缩进的 JSON。 参数说明: <code>const char* path</code> — 目标文件路径 <code>const cJSON* json</code> — 待序列化的 cJSON 对象 返回值: <code>int</code> — 成功返回 0, 失败返回 -1 详细说明: 用于将分析结果 (<code>build_user_summary</code> 生成的 JSON 树) 写入文件, 供后续处理使用。
008	函数原型: <code>cJSON *load_cf_result_array(const char *path)</code> 所属模块: <code>json_utils</code> 功能说明: 读取 CF API 响应 JSON 文件, 提取 <code>result</code> 字段（必须是数组类型）。先 <code>read_json_file</code>, 然后检查 <code>status="OK"</code> 并提取 <code>result</code> 数组。 参数说明: <code>const char* path</code> — CF API 响应 JSON 文件路径 返回值: <code>cJSON*</code> — 指向 <code>result</code> 数组的指针, 失败返回 NULL 详细说明: 用于处理 <code>user.status</code>、<code>user.rating</code>、<code>contest.list</code> 等返回数组的 API 端点。
009	函数原型: <code>cJSON *load_cf_result_first(const char *path)</code> 所属模块: <code>json_utils</code> 功能说明: 读取 CF API 响应 JSON 文件, 提取 <code>result</code> 对象的第一个元素（适用于 <code>result</code> 是对象的情况）。 参数说明: <code>const char* path</code> — CF API 响应 JSON 文件路径 返回值: <code>cJSON*</code> — 指向 <code>result</code> 对象的指针, 失败返回 NULL 详细说明: 用于处理 <code>user.info</code> 端点, 其 <code>result</code> 是一个对象而非数组。

- 010** 函数原型: `int ensure_directory(const char *path)`
所属模块: `json_utils`
功能说明: 跨平台创建目录。在 Windows 上调用 `_mkdir`, 在 Linux 上调用 `mkdir (mode 0755)`。若目录已存在则直接返回成功。
参数说明:
`const char* path` — 待创建的目录路径
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 用于在程序启动时创建 `data/` 和 `output/` 目录, 确保后续文件操作有正确的目录结构。
- 011** 函数原型: `char *cf_http_get(const char *url)`
所属模块: `cf_api`
功能说明: 使用 `libcurl` 发送 HTTP GET 请求, 返回服务器响应的完整内容 (`malloc` 分配)。设置 `User-Agent`、超时时间 (10 秒)、SSL 选项等。
参数说明:
`const char* url` — 完整的 API URL (含查询参数)
返回值: `char*` — `malloc` 分配的响应字符串, 失败返回 `NULL`
详细说明: 内部定义 `MemoryBlock` 结构体和 `write_callback` 函数, 用于动态接收响应数据。调用者负责 `free` 返回的字符串。
- 012** 函数原型: `int fetch_and_save(const char *url, const char *path, RateLimiter *rl)`
所属模块: `cf_api`
功能说明: 通用 API 数据获取与缓存函数: 调用 `rate_limiter_wait` 等待 → `cf_http_get` 请求 → `rate_limiter_tick` 记录 → 将响应写入文件。
参数说明:
`const char* url` — CF API 请求 URL
`const char* path` — 缓存文件保存路径
`RateLimiter* rl` — 指向限速器的指针, 控制请求频率
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 这是所有 CF API 端点函数的底层调用基础, 封装了"限速-请求-缓存"的通用流程。
- 013** 函数原型: `int cf_fetch_user_info(const char *handle, const char *path, RateLimiter *rl)`
所属模块: `cf_api`
功能说明: 获取用户基本信息。调用 `user.info?handles={handle}` 端点, 返回的数据包含 `handle`、`rating`、`maxRating`、`rank`、`avatar` 等字段。
参数说明:
`const char* handle` — Codeforces 用户名
`const char* path` — 缓存文件保存路径
`RateLimiter* rl` — 指向限速器的指针
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 使用 `snprintf` 构造 URL 并调用 `fetch_and_save`。

014 函数原型: `int cf_fetch_user_rating(const char *handle, const char *path, RateLimiter *rl)`
所属模块: `cf_api`
功能说明: 获取用户 Rating 变化历史。调用 `user.rating?handle={handle}` 端点, 返回所有 Rated 比赛的记录 (含 rating 变化、比赛名次等)。
参数说明:
`const char* handle` — Codeforces 用户名
`const char* path` — 缓存文件保存路径
`RateLimiter* rl` — 指向限速器的指针
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 这是获取用户比赛参与历史的主要途径。

015 函数原型: `int cf_fetch_user_status(const char *handle, const char *path, RateLimiter *rl)`
所属模块: `cf_api`
功能说明: 获取用户全部提交记录。调用 `user.status?handle={handle}` 端点, 返回所有提交的详细信息 (含题目 ID、提交时间、判定结果等)。
参数说明:
`const char* handle` — Codeforces 用户名
`const char* path` — 缓存文件保存路径
`RateLimiter* rl` — 指向限速器的指针
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 返回的数据量大, 包含用户在该 CF 账号上的所有提交, 用于赛时/补题判定和 AC 统计。

016 函数原型: `int cf_fetch_contest_list(const char *path, RateLimiter *rl)`
所属模块: `cf_api`
功能说明: 获取 CF 官方比赛列表。调用 `contest.list?gym=false` 端点, 排除训练赛 (gym), 只返回正式比赛。
参数说明:
`const char* path` — 缓存文件保存路径
`RateLimiter* rl` — 指向限速器的指针
返回值: `int` — 成功返回 0, 失败返回 -1
详细说明: 比赛列表是所有用户共享的数据, 只需获取一次。包含每场比赛的 `id`、`name`、`startTimeSeconds`、`durationSeconds` 等。

017	函数原型: <code>int cf_fetch_contest_standings(int contest_id, const char *handle, const char *path, RateLimiter *rl)</code> 所属模块: <code>cf_api</code> 功能说明: 获取用户在指定比赛中的排行榜数据。调用 <code>contest_standings?contestId={id}&handles={handle}&showUnofficial=true</code> 端点。 参数说明: <code>int contest_id</code> — 比赛 ID <code>const char* handle</code> — Codeforces 用户名 <code>const char* path</code> — 缓存文件保存路径 <code>RateLimiter* rl</code> — 指向限速器的指针 返回值: <code>int</code> — 成功返回 0, 失败返回 -1 详细说明: 用于获取每场比赛的详细排名信息、每道题目的得分和提交情况。受 CF API 频率限制, 仅对最近最多 50 场比赛调用。
018	函数原型: <code>const char *rating_color(int rating)</code> 所属模块: <code>analyzer</code> 功能说明: 根据 CF Rating 值返回对应的颜色十六进制字符串。实现 CF 官方 10 级颜色映射体系。 参数说明: <code>int rating</code> — 用户的 CF Rating 整数值 返回值: <code>const char*</code> — 颜色十六进制字符串 (如 <code>"#FF0000"</code>) 详细说明: 映射关系: 0-1199(灰)、1200-1399(绿)、1400-1599(青)、1600-1899(蓝)、1900-2099(紫)、2100-2299(橙)、2300-2399(橙)、2400-2599(红)、2600-2999(深红)、3000+(黑)。使用多级 if-else 判断实现。
019	函数原型: <code>const char *rating_title(int rating)</code> 所属模块: <code>analyzer</code> 功能说明: 根据 CF Rating 值返回对应的用户头衔名称字符串。 参数说明: <code>int rating</code> — 用户的 CF Rating 整数值 返回值: <code>const char*</code> — 头衔名称 (如 <code>"Grandmaster"</code>) 详细说明: 与 <code>rating_color</code> 共享相同的 10 级分界值。头衔包括: Newbie、Pupil、Specialist、Expert、Candidate Master、Master、International Master、Grandmaster、International Grandmaster、Legendary Grandmaster。

020 函数原型: cJSON *build_user_summary(const char *handle, const char *data_dir, const char *output_dir, int max_standings)

所属模块: analyzer

功能说明: 核心分析函数。读取用户的所有缓存数据, 构建完整的分析结果 JSON 树。包含: 用户基本信息、统计数据、比赛历史 (含每道题目得分)、赛时/补题标记、AC 数量直方图 (全部/年度/180天/30天四个时间窗口)。

参数说明:

const char* handle — Codeforces 用户名

const char* data_dir — 缓存数据目录路径

const char* output_dir — 输出目录 (预留参数)

int max_standings — 最大 standings 获取数量 (限制 API 调用次数)

返回值: cJSON* — 包含完整分析结果的 JSON 对象树

详细说明: 关键算法: 1) 赛时/补题判定: 从 contest.list 获取比赛 durationSeconds, 计算比赛结束时间, 对比提交时间, 若首次 AC 时间 $\leq$ 比赛结束时间则为赛时通过, 否则为补题。2) AC 直方图: 遍历 user.status 中的 AC 提交, 按题目 rating 分桶 (每 100 分一档), 分别统计全部、近一年、近 180 天、近 30 天四个时间窗口。3) 180 天活跃度: 基于最近 180 天内的比赛参与次数和表现, 计算 maxRating 变化。

021 函数原型: int generate_html(const char *template_path, const char *output_path, const char *title, cJSON *data)

所属模块: htmlgen

功能说明: 生成单个 HTML 文件。读取 HTML 模板, 将 {{TITLE}} 和 {{DATA_JSON}} 占位符替换为实际数据, 生成自包含的可视化报告。

参数说明:

const char* template_path — HTML 模板文件路径

const char* output_path — 输出 HTML 文件路径

const char* title — 页面标题

cJSON* data — cJSON 数据对象 (将被序列化为 JSON 嵌入页面 JS 变量)

返回值: int — 成功返回 0, 失败返回 -1

详细说明: 模板在 web/template.html 中定义, 包含完整的 HTML 结构、CSS 样式和 ECharts 初始化代码。cJSON_Print 将数据序列化为 JSON 字符串后嵌入 <script> 标签。

022 函数原型: int generate_all_html(cJSON *users, const char *template_path, const char *output_dir)

所属模块: htmlgen

功能说明: 生成所有 HTML 报告。遍历用户数组, 为每个用户调用 generate_html 生成详情页 (report_<handle>.html), 同时生成索引页 (index.html) 展示用户列表和导航。

参数说明:

cJSON* users — 包含所有用户摘要数据的 JSON 数组

const char* template_path — HTML 模板路径

const char* output_dir — 输出目录路径

返回值: int — 全部成功返回 0, 否则返回 -1

详细说明: 索引页面支持用户搜索功能, 提供按 handle 模糊查找的交互。

023

函数原型: `int main(int argc, char *argv[])`所属模块: `main`

功能说明: 程序入口函数, 协调整个工作流程。读取用户列表, 依次执行数据获取、分析、HTML 生成各阶段。

参数说明:

`int argc` — 命令行参数个数`char*[] argv[]` — 命令行参数数组, `argv[1]` 可选指定用户文件路径返回值: `int` — 正常退出返回 0

详细说明: 执行流程: 1) 解析参数, 默认读取 `sample_users.txt` 2) 创建 `data/` 和 `output/` 目录
 3) 初始化 rate limiter 4) 获取比赛列表 (`contest.list`, 所有用户共享) 5) 遍历每个用户: 获取
`user.info` → `user.rating` → `user.status` → 最多 50 场 `contest standings` →
`build_user_summary` 分析 6) 调用 `generate_all_html` 生成全部 HTML 报告 7)
 打印执行摘要 (成功/失败用户数、输出路径)

2.2 函数调用关系图

下图展示了项目中各函数之间的调用关系。

`main` 为程序入口, 调用流程如下:

`main`

```

├─ ensure_directory (创建 data/ 和 output/)
├─ rate_limiter_init (初始化限速器)
├─ cf_fetch_contest_list (获取比赛列表)
├─ 对每个用户循环:
│   ├─ cf_fetch_user_info
│   ├─ cf_fetch_user_rating
│   ├─ cf_fetch_user_status
│   ├─ cf_fetch_contest_standings (最多 50 场)
│   └─ build_user_summary
│       ├─ rating_color / rating_title
│       └─ load_cf_result_array / load_cf_result_first
│           └─ (赛时/补题判定 + AC 直方图统计)
├─ generate_all_html
└─ generate_html (替换模板占位符)
  
```

所有 API 请求均经过:

`fetch_and_save` → `rate_limiter_wait` → `cf_http_get` → `rate_limiter_tick`

读写操作依赖:

`read_text_file` / `write_text_file` / `read_json_file` / `write_json_file`

3 系统演示

3.1 系统各功能运行截图与说明

以下为系统运行过程中的各功能模块截图及说明。

图1：索引页面效果

多用户索引页面（index.html），展示用户列表，支持模糊搜索功能。

图2：用户详情页概览

用户详情页面（report_.html），展示用户基本信息（头像、Rating、头衔、排名等）。

图3：Rating 变化曲线

基于用户参加的所有 Rated 比赛的 Rating 变化数据，使用 ECharts 绘制的折线图。

图4: AC 直方图

按题目 Rating 分桶的 AC

数量统计直方图，支持全部/年度/180天/30天四个时间窗口切换。

图5: 比赛记录表格

用户参赛记录的详细表格，包含比赛名称、时间、排名、得分和 Rating 变化。

图6: 控制台输出

程序运行过程中的控制台日志输出，包含 API 请求状态和数据处理进度。

4 设计体会

4.1 设计过程中的问题及其解决方案

在整个项目的设计与开发过程中，我遇到了一些预料之中及之外的技术难题和挑战。以下是一些主要问题及其对应的思考与解决过程：

问题1：libcurl 环境配置与 SSL 证书问题

在 Windows 环境下配置 libcurl 开发库时，遇到了 SSL 证书验证失败的问题。使用 curl.exe 的 `--ssl-verify disable` 选项解决了证书问题。此外，MSYS2 环境中需要额外安装 libcurl 开发包（mingw-w64-ucrt-x86_64-curl），配置系统 PATH 才能正确链接。

问题2：CF API 频率限制

项目过程中大量对 CF API 进行访问，可能导致 IP 被暂时限制。解决方案是实现 RateLimiter 模块，在连续请求之间加入固定的 2 秒延时（`rate_limiter_wait` 函数），主动控制请求频率，有效降低了被服务器拒绝的概率。

问题3：JSON 数据解析的健壮性

Codeforces API 返回的 JSON 数据结构复杂，存在可选字段或深层嵌套。使用 cJSON 解析时，若字段不存在或类型不匹配容易导致崩溃。解决方案是在解析每个字段前检查是否存在及类型是否匹配（`cJSON_IsString` / `cJSON_IsNumber`），对可选字段赋予默认值，增强容错性。

问题4：赛时/补题判定的算法设计

如何准确区分用户在某场比赛中的每道题是赛时完成还是赛后补题，是项目的关键难点。解决方案是从 `contest.list` 获取每场比赛的 `durationSeconds`，计算比赛结束时间（`startTimeSeconds` + `durationSeconds`），然后遍历 `user.status` 中该场比赛的提交记录，将首次 AC 时间与比赛结束时间比较，实现精确判定。

4.2 设计过程中的错误及其原因分析

在开发过程中，调试和错误排查占据了相当一部分时间。一些典型的错误及其反思如下：

错误1：内存泄漏

初期使用 cJSON 时，频繁调用 `cJSON_Parse` 和 `cJSON_Create*` 系列函数，但未在适当位置调用 `cJSON_Delete` 释放内存，导致程序内存占用持续增长。修正：对每个 `malloc` / `cJSON_Parse` 操作，仔细检查其释放逻辑，确保在各种执行路径下都能正确释放。深刻理解了 C 语言内存管理的“谁申请，谁释放”原则。

错误2：字符串处理不当

程序在拼接 URL、文件名时使用 `sprintf` 但未严格检查缓冲区大小，当源字符串长度超过预期时导致缓冲区溢出和段错误。修正：改用 `snprintf` 并始终明确指定目标缓冲区的最大长度。加强对字符串边界条件的关注。

错误3：API 数据逻辑理解偏差

初期对 `contest standings` API 的参数理解有误，导致无法正确获取用户排名。修正：重新阅读 CF API 文档，使用 `showUnofficial=true` 参数并指定 `handles` 参数，修正了数据提取逻辑。

4.3 体会和收获

通过本次课程设计，我受益匪浅，主要体现在以下几个方面：

1. C 语言编程能力的深化：本项目让我有机会将学到的 C 语言知识（指针、结构体、文件流、动态内存管理等）应用于一个相对复杂的实际问题中，极大地提升了代码实践能力和调试技巧。
2. 对 libcurl 和 cJSON 等第三方库的掌握：通过实际使用 libcurl 进行 HTTP 请求和 cJSON 进行 JSON 解析，我对 C 语言生态中常用的开源库有了初步的了解和掌握，为今后的开发工作积累了宝贵经验。
3. 对 Web API 和数据处理流程的理解：通过实际调用 Codeforces API 并处理返回的 JSON 数据，对 Web API 的工作原理、HTTP 请求/响应模型、JSON 数据格式以及基本的数据清洗、转换和存储流程有了更直观的认识。
4. 问题解决与自主学习能力的提升：在遇到技术难题或 API 使用不明确时，通过查阅官方文档、Stack Overflow、技术博客等资源独立解决问题，锻炼了自主学习和信息检索能力。
5. 文档撰写与总结反思的重要性：编写课程设计文档的过程，也是对整个项目进行梳理、总结和反思的过程。清晰的注释和文档对项目本身和后续维护同样重要。

总而言之，这次大作业不仅是对 C 语言技能的一次全面检验和提升，更是一次宝贵的独立项目经验。虽然过程中遇到了不少困难，但克服它们所带来的成就感和学到的知识，将对我未来的学习和发展产生积极而深远的影响。